SOFTAIMS · PERFORMANCE TESTING

k6 or JMeter?

We installed both. We ran both. We measured both.
Here is what we found, in plain words.

Real screenshots · real numbers September 2026

THE SHORT ANSWER

Use k6 for new work. Keep JMeter for old protocols.

USE THIS BY DEFAULT

k6

For web APIs, cloud apps, and any test that runs automatically when code changes.

- Uses **2.6× less memory**
- Gives an answer **closer to the truth**
- Can **stop a bad build by itself**
- Tests are code a teammate can read

KEEP THIS TOO

JMeter

For things k6 cannot talk to at all.

- Databases, message queues, mail servers
- Tests we already have that work fine
- Teammates who need buttons, not code
- **Nothing gets rewritten**

IN ONE LINE

We are not replacing JMeter. We are choosing what **new** work starts with.

THE ONE IDEA TO UNDERSTAND FIRST

500 users does not mean 500 requests

PEOPLE SIGNED UP
5,000
Most are not using the app right now

USING IT RIGHT NOW
500
App open on their screen

CALLS HITTING THE SERVER
50/sec
This is the real load

Why the numbers drop so much

```
Each person taps about 6 times a minute.
In between, they read the screen.

500 people × 6 taps   = 3,000 a minute
3,000 ÷ 60 seconds    = 50 a second
```

And at any single moment, only about **10 calls** are actually being handled. The rest of the people are reading.

WHY THIS MATTERS BEFORE ANYTHING ELSE

If you test 500 calls a second instead of 50, you are testing **ten times the real traffic**. The app fails, and you report a problem that does not exist.

Getting this wrong matters far more than which tool you pick.

WHAT WE DID

We gave both tools exactly the same job

The trick that makes this fair

We built a small server that **always takes exactly 50 milliseconds** to reply. Never faster, never slower.

So we already knew the right answer. Then we asked each tool to measure it, and checked who got closer.

Same job for both

```
1,000 users
30 seconds to build up
60 seconds holding steady

Same laptop. Same server.
Same everything.
```

What we watched while they ran

- **Memory used** by the tool
- **Workers created** by the tool
- **Processor time** the tool took
- **What each tool reported** as the answer

Checked every half second, for the whole run.

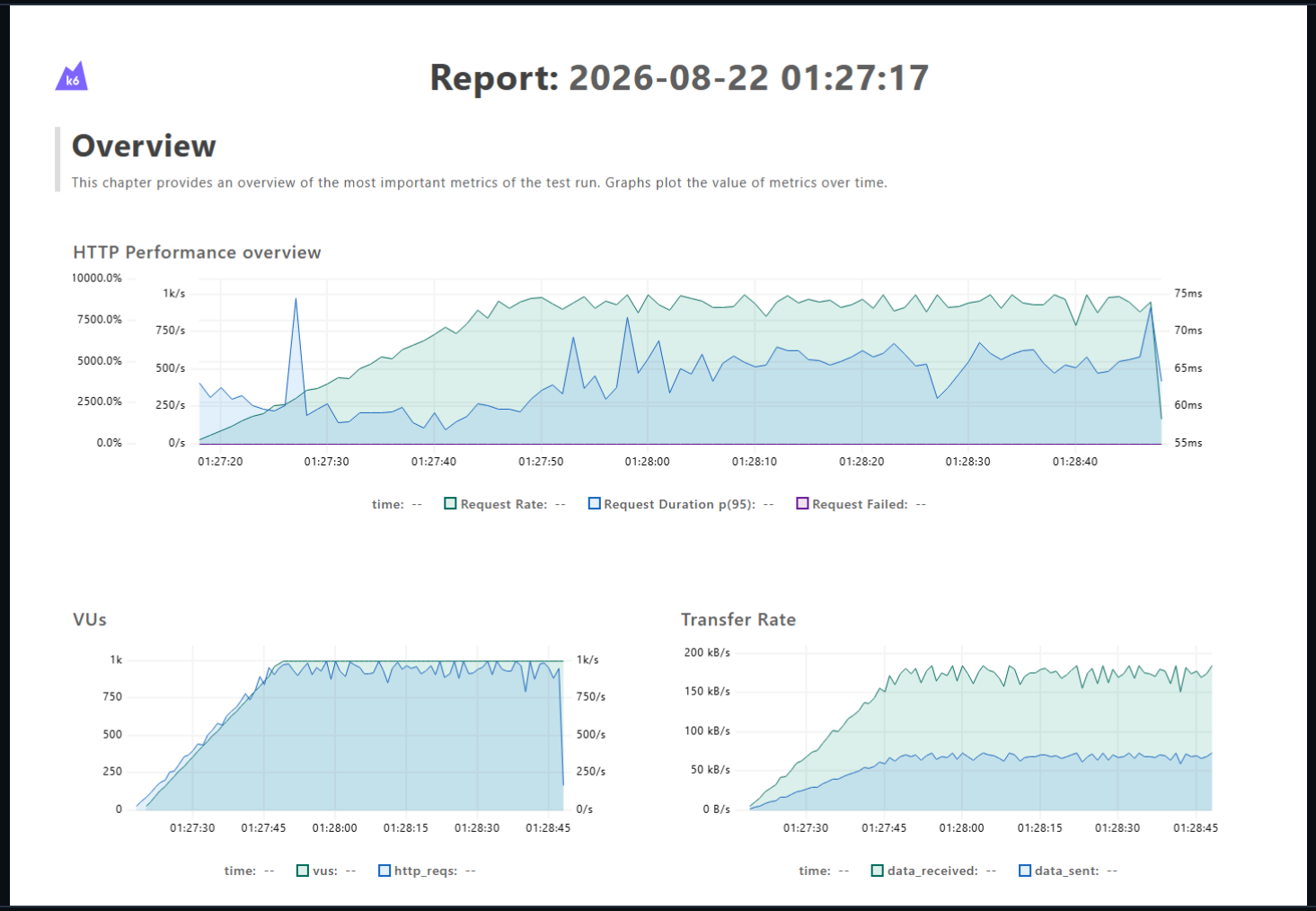
Milliseconds — *thousandths of a second — 50 ms is about how long a blink takes, divided by six*

BOTH TOOLS PASSED

Neither tool crashed. Neither made mistakes. **Zero errors on both sides.** This is a fair fight between two working tools.

REAL SCREENSHOT

What k6 showed us



This is k6's own screen, not something we drew. Left chart: users climbing to **1,000** and holding. The line stays flat and steady — that is a tool in control of what it is doing.

REAL SCREENSHOT

What JMeter showed us

Same test, JMeter's own report. These are the numbers **it printed about itself**:

Statistics

Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent
Total	**69027**	**0**	**0.00%**	**61.66**	**50**	**206**	**63.00**	**69.00**	**71.00**	**76.00**	**775.85**	**140.17**	**90.92**
GET /api	69027	0	0.00%	61.66	50	206	63.00	69.00	71.00	76.00	775.85	140.17	90.92

REQUESTS IT MADE
69,027
with zero failures

LOWEST IT SAW
50 ms
exactly our server's real speed

AVERAGE IT REPORTED
61.66 ms
11.66 ms more than the truth

WHY THIS SCREENSHOT IS OUR STRONGEST EVIDENCE

JMeter's own report says the lowest time it saw was **50 ms**. That proves it reached the very same server. So the gap up to its **61.66 ms** average is **time JMeter added itself** — measured and printed by JMeter, not by us.

SIDE BY SIDE

What the two tools cost the machine

MEMORY
2.6×
k6 used less — 320 MB against 841 MB

WORKERS
69×
k6 used fewer — 15 against 1,037

PROCESSOR
1.34×
k6 used less — 21 sec against 28 sec

Why the worker count is so different

JMeter gives every fake user its own **worker** — *a helper the computer creates and must keep track of — each one costs memory*.

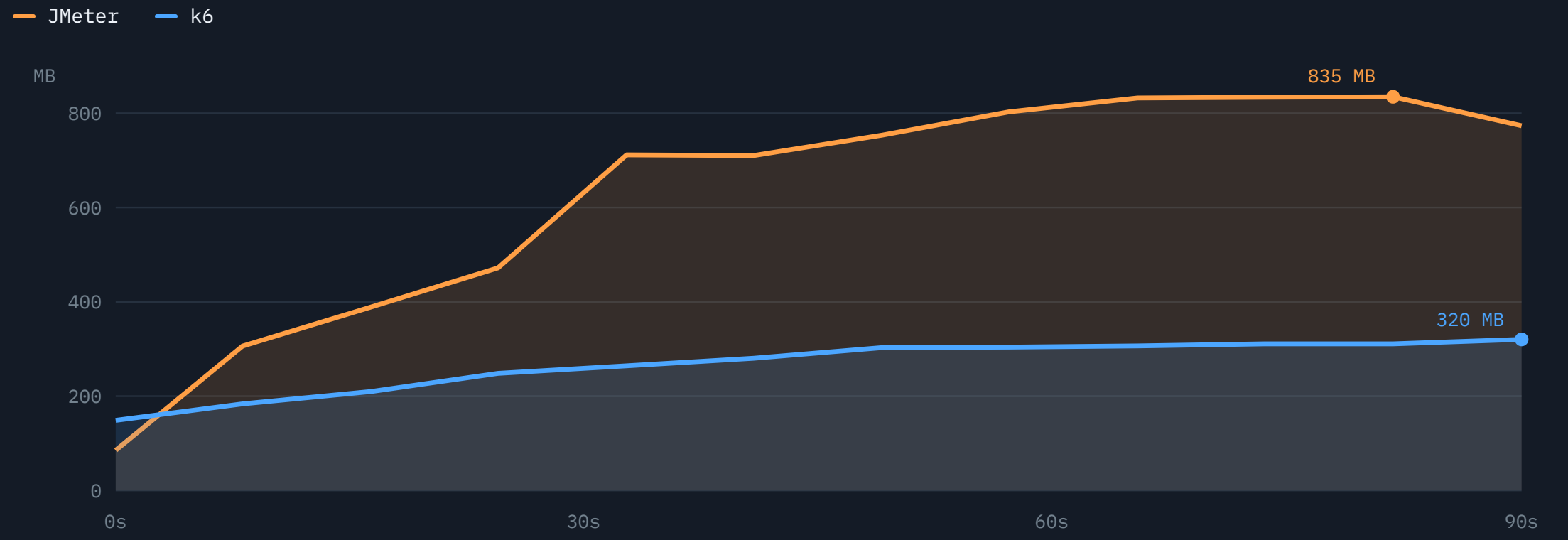
But a user spends almost all its time *waiting*. k6 spotted that, and lets a few workers take turns serving many users.

Same work done. 1,037 helpers against 15.

`Measured on our laptop · Windows 11 · 8 cores · 15.6 GB · files in benchmarks/`

REAL READING

Memory, measured every half second



Every point is a real reading taken from the running programs. JMeter shoots up in the first 10 seconds as it builds its 1,037 workers, then keeps climbing. k6 rises gently and flattens off.

THE IMPORTANT ONE

We knew the right answer. Who got closer?

The server always took exactly **50 milliseconds**. Anything more than that was added by the measuring tool itself.

Extra time added by k6: **6.8 ms**
Extra time added by JMeter: **11.7 ms**

On the worst single request, JMeter said **206 ms** and k6 said **108 ms** — for the same 50 ms server.

THINK OF A THERMOMETER

A thermometer that reads 2 degrees high will tell you a healthy person has a fever. You would not treat the patient. You would change the thermometer.

Both tools run a little warm. **JMeter runs warmer.**

BEING FAIR ABOUT THIS

Neither tool lied. That extra time is real — it is the tool doing its own work. And on a slow API taking 300 ms, a 5 ms difference does not matter. **On a fast API, it decides whether you pass or fail.**

SOMETHING WE FOUND WHILE CHECKING

Three serious security holes — all in the same feature

OFFICIAL ID	SEVERITY	YEAR	WHAT AN ATTACKER COULD DO
CVE-2019-0187	9.8 / 10	2019	**Run any code they liked on your machine**, without a password
CVE-2018-1297	9.8 / 10	2018	Connect to your test machines over an unprotected link
CVE-2018-1287	9.8 / 10	2018	Send unauthorised code to your test machines

All three are in the same place

They are all in the feature you must switch on when **one machine is not enough** and you need several working together.

To use it, JMeter opens network doors between your machines. Those doors are what got attacked.

k6 has **no security holes listed at all** in the official database.

BEING STRAIGHT WITH YOU

All three are already fixed. If you use JMeter 5.1 or newer — and we use 5.6.3 — you are safe from these.

Anyone telling you these are live dangers today is wrong.

SO WHY SHOW THEM?

Because the same feature produced **three top-severity holes**. That tells you something about the design, even after the patches.

Source: National Vulnerability Database (nvd.nist.gov), checked September 2026

A SURPRISE IN JMETER'S OWN MANUAL

Its best feature cannot be used for its main job

APACHE JMETER'S OWN INSTRUCTIONS

JMeter is **not** designed to produce high loads using its screen interface. Doing so “will not only freeze but also consume loads of resources and produce **unreliable load test results**.”

Why that is awkward

The main reason people pick JMeter is the screen you click through — no code needed.

But you may only use that screen to **build** a test. To actually run one, you have to type this:

```
jmeter -n -t test.jmx -l results.jtl
```

So the person who chose JMeter *to avoid the command line* ends up at the command line anyway.

What goes wrong if you ignore it

- Drawing the charts eats the power that should be making traffic
- The screen freezes when there are many users
- Response times come out **higher than they really are**
- It looks exactly like a slow app — so teams hunt for a bug that is not there

Source: jmeter.apache.org/usermanual/best-practices.html

LIVING WITH IT

Can a teammate check your work?

A JMeter test, saved to a file

```
<stringProp name="ThreadGroup
  .num_threads">1000</stringProp>
<elementProp name="HTTPsampler
  .Arguments" elementType="Argum…
<boolProp name="LoopController
  .continue_forever">false</boolProp>
```

About **800 lines** of this, written by the tool for the tool.

A teammate approves it because it ran — **not because they read it**.

A k6 test, saved to a file

```
export default function () {
  const res = http.get(url);
  check(res, {
    'status is 200': r => r.status === 200
  });
  sleep(1);
}
```

About **50 lines**, in the same language much of our team already writes.

Checked the normal way, by someone who **can actually read it**.

THE THREE-YEAR QUESTION

For one test this week, clicking through JMeter is faster. For a set of tests a team keeps for three years, **somebody has to open that file and change it.** That is what we are really choosing.

THE BIGGEST PRACTICAL WIN

The test can stop a bad release on its own

You write the rule once

```
thresholds: {
  http_req_duration: ['p(95)<500'],
  http_req_failed:   ['rate<0.01']
}
```

In plain words: *95 out of every 100 calls must finish in under half a second, and fewer than 1 in 100 may fail.*

If that rule breaks, k6 reports failure by itself. Nothing extra to build.

What happens on every code change

```
someone changes the code
          ↓
    test runs by itself
          ↓
      rule is broken
          ↓
   release is blocked
```

WHY THIS IS WORTH THE MOST

A slowdown caught here costs **ten minutes**. The same slowdown found by a customer costs **a bad weekend**.

Source: grafana.com/docs/k6/latest/using-k6/thresholds/

PUTTING IT IN ONE SUM

Which tool fits *our* work better?

Score = (modern work × its score) + (old work × its score)

Score each tool out of 10 on each kind of work, then weigh it by how much of that work we actually do.

Step 1 — how much of each do we do?

Web APIs, cloud apps, automatic tests	**90%**
Databases, queues, click-only testing	**10%**

This is the only number we chose. Everything else was measured. If you think it is wrong, say so — the next slide shows what happens.

Step 2 — how good is each tool at each?

	K6	JMETER
Modern work	9.5	7.0
Old protocols	2.0	10.0

JMeter gets a clear pass on modern work, and a **perfect 10** on old protocols. k6 gets a 2 there, because it mostly **cannot speak those languages at all**.

WHY THIS SETTLES IT

You pick the percentages. The sum then answers itself — using **your** description of our work, not our opinion.

STEP 3

Now we just do the sum

k6

```
(0.90 × 9.5) + (0.10 × 2.0)
   8.55      +    0.20
             = 8.75
```

JMeter

```
(0.90 × 7.0) + (0.10 × 10.0)
   6.30      +    1.00
             = 7.30
```

RESULT

8.75 vs **7.30**

k6 comes out ahead by **1.45 points**. Not because JMeter is bad — but because **90% of what we do sits where k6 is stronger**.

CHECK IT YOURSELF

Nine tenths of nine and a half, plus one tenth of two. **Nothing is hidden.** Change any number and the sum still works.

THE FAIR TEST

When would this sum choose JMeter instead?

A sum that can only ever say “k6” would be a sales pitch. Here is exactly where it changes its mind.

IF OLD-PROTOCOL WORK IS…	K6	JMETER	WINNER
0% of what we do	9.50	7.00	k6
10% ← us today	8.75	7.30	k6
Just under 1 in 4	7.71	7.71	**Dead heat**
35%	6.88	8.05	JMeter
50%	5.75	8.50	JMeter
All of it	2.00	10.00	JMeter

THE TIPPING POINT

1 in 4

If more than a quarter of our work needs old protocols, JMeter becomes the better choice

SO THE WHOLE THING COMES DOWN TO ONE QUESTION

Is more than **one job in four** at Softaims a database, queue or mail-server test?

If yes, pick JMeter and we were wrong. If no, the sum picks k6 — and it is not close.

THE OTHER SIDE

Three things JMeter simply does better

If we only listed the bad, the first person to name one of these would win the argument. So here they are first.

THE BIG ONE

It talks to more things

Databases, message queues, directory servers, mail servers, file transfers.

k6 cannot speak most of these at all — no setting, no plugin, no workaround.

REAL

Free across many machines

Need ten machines making traffic together? JMeter does that out of the box, on ordinary servers.

k6 needs a Kubernetes cluster or a paid plan for the same thing.

REAL

Buttons, and stability

A tester who does not write code can build a working test by clicking.

And JMeter **does not break its own features**. k6 version 2 removed things that older tests were using.

SAID PLAINLY

JMeter is a **good, mature, free tool** with twenty years behind it. Nothing here says otherwise. Everything here is about which tool fits *our* work — not which tool is better in general.

WHAT WE WILL DO

The plan

STARTING NOW

New tests are written in k6

- Web APIs, cloud apps, microservices
- Every automatic check that runs when code changes
- Anything where we need a pass/fail gate

Scored 8.75. Uses less memory, gives a closer answer, and can block a bad release by itself.

UNCHANGED

JMeter stays exactly where it is

- Databases, queues, directory and mail servers
- Every test we already have that works
- Teammates who prefer clicking to coding

No migration. Nothing rewritten. Nobody loses the JMeter skills they have.

THE SENTENCE TO REMEMBER

k6 is what we reach for first. JMeter is what we keep for the jobs only it can do.

WHERE EVERY NUMBER CAME FROM

Check anything on these slides

We measured these ourselves

- **The whole test.** 1,000 users against a server we built to always take 50 ms. Memory, workers and processor checked every half second. Both tools finished with zero errors.
- **Our laptop.** Windows 11, 8 cores, 15.6 GB memory. k6 v2.1.0, JMeter 5.6.3, Java 17.
- **The screenshots** on slides 5 and 6 are each tool's own screen from that run — unedited.
- **The chart** on slide 8 is drawn from the real readings, nothing smoothed.
- Everything needed to repeat it is in our repository:
  `benchmarks/ — scripts, raw results, instructions`

Official sources

- **Security holes** — National Vulnerability Database, US government
  `nvd.nist.gov — CVE-2019-0187, CVE-2018-1297, CVE-2018-1287`
- **“Not designed for high loads in GUI mode”** — Apache's own manual
  `jmeter.apache.org/usermanual/best-practices.html`
- **Pass/fail rules** — Grafana k6 documentation
  `grafana.com/docs/k6/latest/using-k6/thresholds/`
- **Release dates and community size** — GitHub, 2 September 2026. k6 last released August 2026; JMeter last released January 2024.

THE ONE NUMBER WE CHOSE

The 90% / 10% split on slide 14 is **our estimate** of the work we do. Everything else was measured or published. That is why slide 16 shows what happens if the estimate is wrong.